



UC Redes de Computadores

Prof. Bruno Kimura

16/01/2025

Trabalho 2: Implementação de Protocolo de Transporte Confiável

Observações importantes:

- O trabalho deverá ser realizado por grupos de no máximo 3 alunos.
- Prazo para entrega: **06/02/2024**.
- Entregáveis no *classroom*:
 - Códigos do emissor e receptor na linguagem C contendo comentários que descrevem a implementação.
 - Topologia de rede no emulador CORE.
 - Relatório: entregar exclusivamente via doc no *classroom* (**NÃO ENVIAR PDF**).
- Requisitos do relatório:
 - Apresentar explicação do trabalho desenvolvido: o que, como e porque foi feito.
 - Descrever as contribuições de cada integrante do grupo no desenvolvimento do trabalho.
 - Apresentar auto-avaliação do grupo sobre o trabalho realizado.
- Gestão de recursos:
 - Dividam de forma equivalente as tarefas entre os integrantes do grupo.
 - Definam etapas e prazos. Por exemplo, numa visão macro, 2 semanas para implementação, 1 semana para experimentos e escrita do relatório.

Implemente um protocolo RDT (*Reliable Data Transfer*) para transferência confiável de dados na Internet.

1) Requisitos da implementação:

- **Linguagem.** O protocolo deve ser implementado na Linguagem C, utilizando a API de Sockets.
- **Encapsulamento.** Sua implementação deve encapsular o tratamento da confiabilidade internamente nas funções `rdt_send()` e `rdt_recv()`.
- **Transmissão.** Necessariamente, o RDT deve operar sobre um canal não confiável, ou seja, no nível mais baixo, a comunicação entre o emissor e receptor deve ser feita sobre um `socket` UDP.
- **Temporização.** Sua implementação deve considerar temporizadores do tipo:
 - **Dinâmico**, como o `TimeoutInterval` no TCP (se for ambicioso, aplique Aprendizado de Máquina).
 - **Estático**, baseado em um valor constante arbitrário.
- **Janela de transmissão.** Sua implementação deve considerar buferização de pacote no emissor e receptor, bem como janela de transmissão através de abordagem como Repetição Seletiva de pacotes. O controle sobre a janela de transmissão deve considerar o ajuste:
 - **Estático**, baseado em um tamanho fixo de janela.
 - **Dinâmico**, baseado em abordagens de AIMD.

2) Requisitos para validação do protocolo

- Valide o protocolo sob uma aplicação cliente-servidor para transmissão de arquivos. Após a transmissão, verifique se o arquivo recebido é o mesmo que o original. Para isso, basta verificar a integridade do arquivo recebido, comparando o valor de hash (e.g., `md5sum`, `sha1sum`) do arquivo original com o valor de *hash* do arquivo recebido.
- Utilize a topologia de rede do Trabalho 1, implementada no emulador CORE, para validar o seu protocolo RDT.
- Defina configurações (perdas, atrasos e erros) no enlace de gargalo para que possam ser detectadas no RDT.
- Compile os fontes na máquina host e depois execute-os nos *containers* do emulador.
- Planeje experimentos considerando diferentes fatores, como abordagens de temporização (estático, dinâmico), abordagens de janelas de transmissão (estática, dinâmica), configurações de rede (taxas de perdas/erros, latência, capacidade dos enlaces de gargalo na rede).
- Verifique métricas de QoS, como vazão e latência, resultantes do seu protocolo. Compare e discuta os resultados experimentais entre as abordagens definidas pelos fatores.



Instituto de Ciência e Tecnologia – ICT
Universidade Federal de São Paulo - UNIFESP





ANEXO I

Informações de Interesse

- Utilize a bibliografia básica da disciplina (*J. Kurose, K. Ross, Redes de Computadores: Uma Abordagem Top-Dow, 5ª edição, 2010*) e os códigos de exemplo desenvolvidos em aula.
- O exemplo de código abaixo configura um temporizador de 5 segundos para o recebimento sobre o descritor `sockfd`. O efeito desse temporizador se dará como o tempo máximo de bloqueio da primitiva `recvfrom()`, por exemplo, na espera do ACK. Nesse caso, após a chamada dessa primitiva, se dentro de 5 segundos a aplicação não receber o ACK, a primitiva irá retornar com erro. A variável `errno` terá valor de `EAGAIN` ou `EWOULDBLOCK`. Portanto, após configurar o temporizador sobre o descritor, a operação de `start_timer` da máquina de estado do emissor será acionada automaticamente sempre que a primitiva `recvfrom()` for chamada na espera de um ACK.

```
struct timeval timeout;  
timeout.tv_sec = 5; // segundos  
timeout.tv_usec = 0; // microssegundos  
// timeout para recebimento  
if (setsockopt (sockfd, SOL_SOCKET, SO_RCVTIMEO, (char *)&timeout, sizeof(timeout)) < 0)  
    perror("setsockopt(..., SO_RCVTIMEO ,...)");
```

- A soma de verificação, `checksum`, está descrita na RFC1071:

<https://tools.ietf.org/html/rfc1071>